

Databricks Unity Catalog vs Snowflake Horizon

Empirical tests of Iceberg fine-grained access-control enforcement across three foreign-engine boundaries: OSS Spark reading a UC-managed table, OSS Spark reading a Snowflake-managed table, and Databricks UC Federation reading a Snowflake-managed table.

Date 2026-05-07
Workspace DBR 16.4 LTS+ (Databricks Unity Catalog Iceberg REST is GA / Public Preview from 16.4; Snowflake Catalog Federation is GA on 13.3+)
External client OSS PySpark 3.5 + `iceberg-spark-runtime-3.5_2.12:1.9.1` + `iceberg-aws-bundle:1.9.1`; Databricks SQL warehouse for federation tests
Auth to Databricks Unity Catalog Bearer token (PAT or OAuth) via the Databricks Unity Catalog Iceberg REST endpoint

1. Question under test

A reference Snowflake Horizon test (`spark_horizon_policy_test.py`) shows that when a Snowflake-managed Iceberg table has a row access policy and column masks attached, an external OSS Spark client connecting via the Polaris REST catalog with vended credentials still receives a fully *governed* view of the table: filtered rows, masked columns, served from Snowflake's data plane.

Question:

“Is the same true for a Databricks-managed Iceberg table accessed from OSS Spark via Unity Catalog's Iceberg REST endpoint? If we attach a Databricks Unity Catalog row filter and column masks, will an external Spark client still get the governed view of the data?”

The short answer is **no**, and the failure mode is materially different from what most reviewers expect. Databricks Unity Catalog does not enforce policy server-side — it cannot, the engine on the other end is OSS Spark reading parquet directly from S3. Instead, the moment any row filter or column mask is attached, Databricks Unity Catalog *scrubs its REST response* so the external client has nothing readable. The net effect is that the table becomes unreadable from any non-Databricks engine.

2. Documented behavior (Databricks)

From the Databricks documentation page “*Row filters and column masks*”, Limitations section:

“You cannot use Iceberg REST catalog or Unity REST APIs to access tables with row filters or column masks.”
— docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks#limitations

And from the Databricks Knowledge Base article on Databricks Unity Catalog FGAC:

“ [UNAUTHORIZED_ACCESS] Path-based access to table <catalog>.<schema>.<table> with row filter or column mask not supported. ... Only name-based access (catalog.schema.table) is allowed.”
— kb.databricks.com – “Unauthorized access exception ... row filters or column masks”

The test below operationally verifies this behavior and characterizes the actual failure mechanism, which is more nuanced than the docs convey.

3. Test environment

Component	Value
Catalog / schema / table	<catalog>.policy_test.policy_test_table
Securable kind	TABLE_DELTA_ICEBERG_MANAGED — Databricks Unity Catalog-managed Iceberg, created with USING ICEBERG
Storage	Databricks Unity Catalog-managed S3 path (workspace bucket, Databricks Unity Catalog owns the layout)
External client	OSS PySpark 3.5 / Java 11, iceberg-spark-runtime-3.5_2.12:1.9.1 + iceberg-aws-bundle:1.9.1
Catalog endpoint	<workspace>/api/2.1/unity-catalog/iceberg-rest
Vended credentials	Header X-Iceberg-Access-Delegation: vended-credentials
Authentication	Bearer token (PAT or OAuth)
Privileges of caller	USE CATALOG , USE SCHEMA , SELECT , EXTERNAL USE SCHEMA on the schema
FGAC functions	row_filter_us_ca , mask_email , mask_ip (SQL UDFs, group-exempt for admins / analytics_admin)

The caller is intentionally *not* a member of `admins` or `analytics_admin`, so the policies actively apply. This was confirmed by the Databricks SQL warehouse query in Phase B (3 rows, masked).

4. Methodology

1. **Phase A — baseline:** create the table, load 8 rows, attach *no* policies. Read the table from Databricks SQL *and* from OSS Spark via Databricks Unity Catalog Iceberg REST. Both should succeed and return the same raw 8 rows.
2. **Apply FGAC:** attach `row_filter_us_ca` on `country`, `mask_email` on `email`, `mask_ip` on `ip_address`.
3. **Phase B — under policy:** re-read the same table from Databricks SQL (expected: 3 rows, masked) *and* from OSS Spark (expected: failure or empty result).
4. **Forensic probe:** for each phase, hit the Databricks Unity Catalog Iceberg REST `loadTable` endpoint directly with `curl` and inspect the JSON response. This is what reveals the *mechanism* of Databricks Unity Catalog’s “fail-secure” behavior.

5. Test data & FGAC functions

```
CREATE TABLE <catalog>.policy_test.policy_test_table (
  user_id      BIGINT,
  email        STRING,
  ip_address   STRING,
  country      STRING,
  event_type   STRING
) USING ICEBERG;

INSERT INTO <catalog>.policy_test.policy_test_table VALUES
(1,'alice@example.com', '192.168.1.10', 'US','login'),
(2,'bob@example.com',   '10.0.0.5',   'US','login'),
(3,'carol@example.com', '172.16.0.20', 'CA','vault_open'),
(4,'dave@example.co.uk', '203.0.113.42', 'UK','login'),
(5,'eve@example.de',    '198.51.100.7', 'DE','failed_login'),
(6,'frank@example.fr',  '192.0.2.55',   'FR','login'),
(7,'grace@example.jp',  '203.0.113.99', 'JP','vault_open'),
(8,'heidi@example.au',  '198.51.100.88', 'AU','login');
```

```
CREATE OR REPLACE FUNCTION row_filter_us_ca(country STRING) RETURNS BOOLEAN
RETURN is_account_group_member('admins')
      OR is_account_group_member('analytics_admin')
      OR country IN ('US','CA');

CREATE OR REPLACE FUNCTION mask_email(email STRING) RETURNS STRING
RETURN CASE
  WHEN is_account_group_member('admins')
    OR is_account_group_member('analytics_admin') THEN email
  WHEN email IS NULL OR instr(email,'@')=0      THEN email
  ELSE concat(substring(email,1,1), '***', substring(email, instr(email,'@'))))
END;

CREATE OR REPLACE FUNCTION mask_ip(ip STRING) RETURNS STRING
RETURN CASE
  WHEN is_account_group_member('admins')
    OR is_account_group_member('analytics_admin') THEN ip
  WHEN ip IS NULL THEN ip
  ELSE concat('***.***.***.', element_at(split(ip,'\\.'),-1))
END;

ALTER TABLE policy_test_table SET ROW FILTER row_filter_us_ca ON (country);
ALTER TABLE policy_test_table ALTER COLUMN email      SET MASK mask_email;
ALTER TABLE policy_test_table ALTER COLUMN ip_address SET MASK mask_ip;
```

6. Results

6.1 Phase A — no policies attached

Databricks SQL warehouse (non-admin caller):

user_id	email	ip_address	country	event_type
1	alice@example.com	192.168.1.10	US	login
2	bob@example.com	10.0.0.5	US	login
3	carol@example.com	172.16.0.20	CA	vault_open
4	dave@example.co.uk	203.0.113.42	UK	login
5	eve@example.de	198.51.100.7	DE	failed_login
6	frank@example.fr	192.0.2.55	FR	login
7	grace@example.jp	203.0.113.99	JP	vault_open
8	heidi@example.au	198.51.100.88	AU	login

OSS Spark via Databricks Unity Catalog Iceberg REST (spark_uc_policy_test.py):

```

+-----+-----+-----+-----+-----+
|user_id|email          |ip_address |country|event_type |
+-----+-----+-----+-----+-----+
|1      |alice@example.com|192.168.1.10|US    |login      |
|2      |bob@example.com  |10.0.0.5    |US    |login      |
|3      |carol@example.com|172.16.0.20 |CA    |vault_open |
|4      |dave@example.co.uk|203.0.113.42|UK    |login      |
|5      |eve@example.de   |198.51.100.7|DE    |failed_login|
|6      |frank@example.fr  |192.0.2.55  |FR    |login      |
|7      |grace@example.jp  |203.0.113.99|JP    |vault_open |
|8      |heidi@example.au  |198.51.100.88|AU    |login      |
+-----+-----+-----+-----+-----+
Rows returned: 8 ; email masked? False ; ip masked? False

```

Forensic probe of the Databricks Unity Catalog Iceberg REST `loadTable` response:

```

config keys      : ['client.region', 's3.access-key-id', 's3.secret-access-key',
                    's3.session-token', 's3.session-token-expires-at-ms']
vended S3 access-key-id : PRESENT
vended S3 session-token : PRESENT
storage-credentials count: 0
snapshot.manifest-list  : 's3://<workspace-bucket>/.../_iceberg/metadata/snap-....avro'
Verdict: READABLE

```

Databricks Unity Catalog vends short-lived S3 credentials in the `config` block and returns the real manifest-list pointer. Spark's Iceberg client uses the credentials to read the manifest list, plan the scan, fetch parquet data files, and return rows.

6.2 Phase B — row filter and column masks attached

Databricks SQL warehouse (same caller):

```

user_id  email          ip_address      country  event_type
1        a***@example.com ***.*.*.*.10    US       login
2        b***@example.com ***.*.*.*.5     US       login
3        c***@example.com ***.*.*.*.20    CA       vault_open

```

Databricks Unity Catalog enforces the row filter and column masks on Databricks compute: 3 of 8 rows survive, `email` and `ip_address` are masked. Databricks Unity Catalog's name-based, in-engine policy enforcement is working as documented.

OSS Spark via Databricks Unity Catalog Iceberg REST (same client, same script):

```

QUERY FAILED: Py4JJavaError: An error occurred while calling o62.showString.
: org.apache.iceberg.exceptions.ValidationException:
  Invalid S3 URI, cannot determine scheme:
  at org.apache.iceberg.aws.s3.S3URI.<init>(S3URI.java:75)
  at org.apache.iceberg.aws.s3.S3InputFile.fromLocation(S3InputFile.java:41)
  at org.apache.iceberg.aws.s3.S3FileIO.newInputFile(S3FileIO.java:176)
  at org.apache.iceberg.BaseSnapshot.cacheManifests(BaseSnapshot.java:176)
  at org.apache.iceberg.BaseSnapshot.deleteManifests(BaseSnapshot.java:210)
  at org.apache.iceberg.BaseDistributedDataScan.findMatchingDeleteManifests ...

```

Forensic probe of the Databricks Unity Catalog Iceberg REST `loadTable` response:

```

config keys      : []
vended S3 access-key-id : ABSENT
vended S3 session-token : ABSENT
storage-credentials count: 0
snapshot.manifest-list  : ''
Verdict: BLOCKED (no creds and/or empty manifest-list)

```

This is the punchline. The HTTP 200 response still carries a `metadata-location` pointer, but every actionable field has been redacted by Databricks Unity Catalog:

- `config` is an empty object – **no vended S3 credentials**.
- `storage-credentials` is an empty array.
- The latest snapshot's `manifest-list` is the **empty string** `""`.

The Iceberg client on the OSS Spark side trusts the response, sees an empty-string manifest-list, and tries to construct an S3URI from it — which fails synchronously with `Invalid S3 URI, cannot determine scheme:` (note the trailing colon and empty value). That is the exception you see at runtime.

7. Mechanism — how Databricks Unity Catalog blocks external readers

Databricks Unity Catalog has no way to enforce a row filter or a column mask on an OSS Spark client. The client connects to a REST endpoint, gets a metadata pointer plus credentials, and from that point on reads parquet directly from S3 without Databricks Unity Catalog ever seeing the read. Databricks Unity Catalog has only one moment of control — the `loadTable` response — and on policed tables it uses that moment to remove every piece of information the client needs:

Field in <code>loadTable</code> response	Phase A (no policies)	Phase B (policies on)
<code>config.s3.access-key-id</code>	PRESENT	ABSENT
<code>config.s3.session-token</code>	PRESENT	ABSENT
<code>config.client.region</code>	PRESENT	ABSENT
<code>storage-credentials[]</code>	(empty — creds in <code>config</code>)	empty
<code>metadata.snapshots[-1].manifest-list</code>	real S3 path (265 chars)	empty string <code>""</code>
<code>metadata-location</code>	real S3 path	real S3 path (kept, but useless without creds)

Key observation. Databricks Unity Catalog's policy enforcement model for external engines is “refuse to give them anything they can use”. It is *not* “return data with the policy applied,” which is what Snowflake Horizon does. Functionally the table becomes unreadable from any non-Databricks Iceberg client (OSS Spark, Trino, Flink, Pylceberg, Snowflake catalog-linked databases, etc.) for as long as the policy is attached.

8. Side-by-side comparison with Snowflake Horizon

Capability	Snowflake Horizon (Polaris REST)	Databricks Unity Catalog (Iceberg REST)
Native compute reads policed table	Filtered + masked rows served	Filtered + masked rows served
External OSS Spark reads policed table	Filtered + masked rows served (server-side enforcement)	Refused — empty creds, blank manifest-list, client errors
External engine receives vended credentials	Yes — scoped to the policy's filtered/masked view	No — config: {}
External engine receives a real manifest-list	Yes — points at the filtered/masked snapshot	No — manifest-list: ""
Governance posture	Open Iceberg + governed FGAC across engines	Open Iceberg + FGAC only when callers stay on Databricks compute
Net result for multi-engine architectures	Single governance plane, all engines see the same governed view	Either-or: governance or openness, not both, on a per-table basis

9. Inverse boundary — Snowflake-policed table read from Databricks via UC Federation

Once the OSS-Spark direction has been characterized for both platforms, the obvious follow-up is the inverse case: *Snowflake* is the governance plane, and *Databricks* is the foreign reader. UC offers two integrations into Snowflake; they look superficially similar but take very different paths to the data.

Path	Where the query runs	What UC reads	Where the policy is evaluated
Query Federation (CONNECTION TYPE = SNOWFLAKE, FOREIGN CATALOG without <code>authorized_paths</code>)	Snowflake virtual warehouse	Result set returned over JDBC	Snowflake's SQL engine, at query time
Catalog Federation (same connection, FOREIGN CATALOG with <code>authorized_paths</code>)	Databricks compute (Photon / Spark)	Parquet files directly from S3, reached via UC's storage credential	Nowhere — Snowflake's query engine is not in the path; row filters and column masks are query-time constructs that were never serialized into the parquet files

The two paths produce dramatically different observable results when the same Snowflake table has FGAC policies attached and the federation connection is bound to a non-admin Snowflake role.

9.1 Setup

Snowflake side: a managed Iceberg table at `<db>.<schema>.policy_test_table` with the same 8 rows and the same row-access plus column-mask policies as the Polaris test in section 6. The connection's Snowflake role is `<restricted_role>`, intentionally *not* in the policies' admin-exempt list.

Databricks side: two foreign catalogs, both bound to the same connection.

```

-- Path 1: Query Federation (no authorized_paths).
CREATE FOREIGN CATALOG sf_query_fed USING CONNECTION sf_conn
  OPTIONS ('database' '<db>');

-- Path 2: Catalog Federation (authorized_paths covers the table's S3 prefix).
CREATE FOREIGN CATALOG sf_catalog_fed USING CONNECTION sf_conn
  OPTIONS (
    'database'      '<db>',
    'authorized_paths' 's3://<snowflake_external_volume_bucket>/<prefix>/'
  );

```

`DESCRIBE EXTENDED` confirms which mode is actually in effect for each catalog:

Catalog	Provider	Type	Interpretation
<code>sf_query_fed. <schema>.policy_test_table</code>	snowflake	FOREIGN	JDBC pushdown to Snowflake compute
<code>sf_catalog_fed. <schema>.policy_test_table</code>	iceberg	EXTERNAL	UC reads parquet directly from S3 using the storage credential bound to <code>authorized_paths</code>

9.2 Probe queries

Snowflake-side baseline under `<restricted_role>` — the result we expect any trustworthy reader of the table to see:

```

USER_ID  EMAIL                IP_ADDRESS      COUNTRY  EVENT_TYPE
1        a***@example.com    ***.***.***.10 US        login
2        b***@example.com    ***.***.***.5  US        login
3        c***@example.com    ***.***.***.20 CA        vault_open

```

Query Federation — Databricks reads `sf_query_fed.<schema>.policy_test_table`:

```

USER_ID  EMAIL                IP_ADDRESS      COUNTRY  EVENT_TYPE
1        a***@example.com    ***.***.***.10 US        login
2        b***@example.com    ***.***.***.5  US        login
3        c***@example.com    ***.***.***.20 CA        vault_open

```

Identical to the Snowflake baseline. Snowflake's row filter and masking policies are honored. The query was rewritten and pushed down over JDBC; Snowflake's SQL engine evaluated the policies before returning rows to Databricks.

Catalog Federation — Databricks reads `sf_catalog_fed.<schema>.policy_test_table`:

```

USER_ID  EMAIL                IP_ADDRESS      COUNTRY  EVENT_TYPE
1        alice@example.com    192.168.1.10   US        login
2        bob@example.com      10.0.0.5        US        login
3        carol@example.com    172.16.0.20     CA        vault_open
4        dave@example.co.uk   203.0.113.42   UK        login
5        eve@example.de       198.51.100.7    DE        failed_login
6        frank@example.fr     192.0.2.55      FR        login
7        grace@example.jp     203.0.113.99   JP        vault_open
8        heidi@example.au     198.51.100.88  AU        login

```

Every guarantee Snowflake offers on this table for non-admin roles has been bypassed. The row-access policy is gone (UK / DE / FR / JP / AU rows are present); both column-mask policies are gone (8/8 unmasked emails, 8/8 unmasked IPs). The Snowflake table itself has not changed; only Databricks's access path has.

Numerical sanity:

Query	Query Federation	Catalog Federation
<code>COUNT(*)</code>	3	8
<code>COUNT(DISTINCT country_code)</code>	2 (US, CA)	7 (US, CA, UK, DE, FR, JP, AU)
Unmasked emails	0/3	8/8
Unmasked IPs	0/3	8/8
<code>WHERE country_code NOT IN ('US', 'CA')</code>	0 rows (filter active)	5 rows (filter inactive)

9.3 Mechanism

1. Snowflake row-access policies and masking policies are *query-time* rewrites in Snowflake's SQL engine. They are not row-level deletions, not column-level encryption, not encoded into the parquet files.
2. The parquet files written by Snowflake under the table's external volume contain the complete, raw, ungoverned rows. They are exactly what an admin would see in-warehouse.
3. When asked for the `metadata.json` location, Snowflake returns the unredacted path even under non-admin roles. This was verified directly with `SELECT SYSTEM$GET_ICEBERG_TABLE_INFORMATION('<table>')` under both `ACCOUNTADMIN` and `<restricted_role>`: identical paths.
4. Catalog Federation hands that path to UC, which then reads the parquet directly using its own storage credential. Snowflake's query engine is no longer in the loop, so neither is its FGAC enforcement.

This is the architectural inverse of UC's own behavior. Databricks Unity Catalog scrubs `loadTable` for policed tables and refuses to hand the raw parquet to external readers (fail-secure, section 7). Snowflake's metadata service does not scrub; it trusts the engine to honor the policy at query time, which works for Snowflake compute but not for an engine that bypasses the engine.

9.4 Fallback mode is a safety net, not the rule

Databricks documents a fallback rule for Catalog Federation: tables whose metadata location is outside the declared table location, or that use unsupported URI schemes / special characters, silently fall back to Query Federation. When fallback kicks in, the query goes back over JDBC and Snowflake's policies enforce again. So you may see policy enforcement even on a foreign catalog that you set up for catalog federation — and conclude, incorrectly, that the path is safe. It is not; you got lucky on the table layout. **Always verify with** `DESCRIBE EXTENDED`; only `Provider: iceberg` means you are actually exercising the catalog-federation read path.

10. Implications

- Any architecture that relies on Databricks Unity Catalog FGAC for sensitive tables and also expects external engines (OSS Spark on EMR, Trino, Flink, Pylceberg, Snowflake via catalog-linked databases, ad-hoc Pylceberg in scripts) to read those same tables is structurally incompatible with Databricks Unity Catalog today. The two requirements cannot both be satisfied.
- The failure mode is silent at the HTTP level: `loadTable` returns 200 OK. Operators looking at REST traffic will not see a 403, they will see a successful response that just happens to contain no usable data. Diagnostics will surface only on the client (e.g. `Invalid S3 URI`) and will look like a client-side bug rather than an authorization decision.
- Workarounds (use a dynamic view; copy the table to an unmasked external location; expose only Databricks compute to consumers) re-introduce the duplication and surface area that motivated putting FGAC on the base table in the first place.
- By contrast, Snowflake Horizon's model treats the data plane as the enforcement plane *for engines that go through Snowflake compute* (native, OSS Spark via Polaris, Databricks Query Federation). Any of these clients gets the governed view of the table.

- Snowflake’s open posture has an edge case: any access path that hands raw S3 reads to a foreign engine — specifically Databricks Catalog Federation, but also any other reader granted access to the underlying bucket — bypasses the row-access and masking policies entirely. Customers running Snowflake FGAC and considering Databricks as a reader should default to Query Federation, treat Catalog Federation as unsuitable for any table whose security depends on row filters or column masks, and audit existing Catalog Federation foreign catalogs to make sure no policed tables are exposed.

The test bundle on GitHub contains three symmetric scenarios:

Databricks side (databricks/)	Snowflake side (snowflake/)	Snowflake-from-Databricks (snowflake/databricks_federation/)
01_setup_table_and_data.sql	01_setup_table_and_data.sql	01_databricks_query_federation.sql
02_apply_row_filter_and_masks.sql	02_apply_policies.sql	02_databricks_catalog_federation.sql
03_drop_policies.sql	03_drop_policies.sql	03_test_queries.sql
spark_uc_policy_test.py	spark_horizon_policy_test.py	findings.md
probe_uc_iceberg_rest.sh	probe_polaris_iceberg_rest.sh	README.md

Repro outline (Databricks side; the Snowflake side is in the companion `BLOG.md` and has equivalent shape):

```
# Step 1 (Databricks SQL editor):
\i databricks/01_setup_table_and_data.sql

# Step 2 (laptop):
export DATABRICKS_HOST=https://<your-workspace>.cloud.databricks.com
export DATABRICKS_TOKEN=<personal-access-token-or-oauth-bearer>
export UC_CATALOG=<your-catalog>
export JAVA_HOME=$(/usr/libexec/java_home -v 11)

python3 databricks/spark_uc_policy_test.py      # Phase A: 8 rows, raw
./databricks/probe_uc_iceberg_rest.sh           # Phase A: READABLE

# Step 3 (Databricks SQL editor):
\i databricks/02_apply_row_filter_and_masks.sql

python3 databricks/spark_uc_policy_test.py      # Phase B: ValidationException: Invalid S3 URI
./databricks/probe_uc_iceberg_rest.sh           # Phase B: BLOCKED (no creds, empty manifest-list)

# Optional cleanup:
\i databricks/03_drop_policies.sql
```

Snowflake-from-Databricks federation outline:

```
# Snowflake side already has the policied table from snowflake/01_*.sql + 02_*.sql.

# Databricks side, in a SQL editor as a user with CREATE CONNECTION:
# - edit placeholders in
#   snowflake/databricks_federation/01_databricks_query_federation.sql
# and run it (creates CONNECTION + FOREIGN CATALOG, no authorized_paths).
# - edit placeholders in
#   snowflake/databricks_federation/02_databricks_catalog_federation.sql
# and run it (FOREIGN CATALOG with authorized_paths).
# - run
#   snowflake/databricks_federation/03_test_queries.sql
# against both catalogs and compare row counts, country histograms,
# and the email / ip_address columns side by side.

# DESCRIBE EXTENDED tells you whether you're really exercising catalog
# federation (Provider: iceberg, FGAC bypassed) or whether UC silently fell
# back to query federation for that specific table layout
# (Provider: snowflake, FGAC enforced via the JDBC pushdown safety net).
```

12. References

- Databricks — Row filters and column masks — Limitations: docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks#limitations
- Databricks — Access Databricks tables from Apache Iceberg clients: docs.databricks.com/aws/en/external-access/iceberg
- Databricks — Unity Catalog credential vending for external system access: docs.databricks.com/en/external-access/credential-vending
- Databricks KB — Unauthorized access exception ... row filters or column masks: kb.databricks.com/en_US/delta/unauthorized-access-exception-when-trying-to-access-a-unity-catalog-table-with-row-filters-or-column-masks
- Apache Iceberg — Issue #10909 “Support row filter & column masking in REST spec” (closed: *not_planned*): github.com/apache/iceberg/issues/10909
- Databricks — What is catalog federation?: docs.databricks.com/aws/en/query-federation/catalog-federation
- Databricks — Enable Snowflake catalog federation: docs.databricks.com/aws/en/query-federation/snowflake-catalog-federation
- Snowflake — Row access policies: docs.snowflake.com/en/user-guide/security-row-intro

All command output and REST payloads quoted in this document were captured live from a Databricks workspace running the test bundle described in section 10.